

TEST PLAN

Team members

1. Philimon Masangwa
2. Mary Kibet
3. Sarafina Barnabas
4. Angela Loishooki

SELECTED PROJECT

Vehicle Rental Management System

CLASS BEING TESTED

Customer Class

TEST CASES - BEFORE TESTING

Test ID	Method Being Tested	Scenario Description	Inputs	Expected Outputs/Behaviours
TC1	__init__	Creating a customer with all valid, non-empty fields	customer_id="C001", customer_name="Lemon Masangwa", phone_number="58589467"	Customer object is created successfully, email and driver_license default set to None
TC2	Display_details	Displaying a valid customer's details after creation	A customer object created with TC1's data	Print all five fields, with email and license shown as None if not provided
TC3	__init__	Edge case: customer_id containing only whitespace, not a true empty string	Customer_id =" " (empty white space), other required fields valid	Object is created without error, validation only checks for a completely empty string so whitespace only input passes through
TC4	__init__	Invalid case: customer_id left	Customer_id ="", other required	Raises ValueError

		completely empty	fields valid	(Customer ID cannot be empty.); No object is created
TC5	__init__	Invalid case: phone_number left completely empty	Phone_number = " ", other fields valid	Raises ValueError (Phone number cannot be empty.); no object is created

CLASS BEING TESTED

Vehicle Class

TEST CASES - BEFORE TESTING

Test ID	Method Being Tested	Scenario Description	Inputs	Expected Outputs/Behaviours
TC1	__init__ / display_details	Creating a vehicle with valid data and displaying it	vehicle_id="f0001", registration="bd234", make="Ford", model="focus", daily_rate=60	Vehicle created; display_details() prints all fields with status; Available
TC2	Rent_a_vehicle / return_a_vehicle	Renting then returning a vehicle updates its availability correctly	Call rent_a_vehicle(), then return_a_vehicle() on TC1's vehicle	Is_available goes True-False after renting, then False-True after returning
TC3	add_vehicle (main.py) vs vehicle.__init__ directly	Confirms where negative rate protection is through the real app menu against bypassing it	Through the app: enter -50 when adding a vehicle. Bypassing the app: call Vehicle (daily_rate=-50.0) directly in code	Through the app: rejected with "Rate must be greater than zero.", no vehicle created — verified working correctly. Bypassing the app: no error, daily_rate stored as -50.0,

				since the check lives in main.py's add_vehicle(), not inside the Vehicle class itself
TC4	search_for_vehicle	Searching by ID, by registration and with no match	search_term="V001", search_term="BC865", search_term="ZZ759"	Returns True (and prints details) for the first two, returns False for the non-matching search
TC5	to_dictionary	Confirming the dictionary used for saving to file matches the object's actual data	A vehicle object with known values	Returns a dictionary with all six fields matching the object's current attributes exactly

CLASS BEING TESTED

Rental Class

TEST CASES - BEFORE TESTING

Test ID	Method Being Tested	Scenario Description	Inputs	Expected Outputs/Behaviours
TC1	__init__ / calculate_cost	Creating a rental and calculating its cost for a normal multi-day rental	rental_id="R001", vehicle_id="v0234", customer_id="c001", days= 3, daily_rate passed to calculate_cost= 50.0	total_cost is correctly calculated as 150 (days * rate)
TC2	display_details	Displaying an active vs a completed rental	Same rental as TC1, first with is_active = True, then set to False	Prints "Active" in the first case and "completed" in the second

TC3	calculate_cost	Edge case: a rental of 0 days	Days = 0, daily_rate = 50.0	total_cost calculates to 0.0 (no error, but a 0 day rental may not be meaningful, worth deciding whether main.py should ever allow this)
TC4	rent_vehicle (main.py) vs Rental. __init__ directly	Confirms where negative days protection actually lives: through the real app menu vs bypassing it	Through the app: enter -2 when renting a vehicle. Bypassing the app: call Rental(days=-2) directly in code	Through the app: rejected with "Days must be at least 1.", no rental created, verified working correctly. Bypassing the app: no error, days=-2 stored, cost calculates to -100.0, since the check lives in main.py's rent_vehicle(), not inside the Rental class itself
TC5	to_dictionary	Confirming the dictionary used for saving to file matches the object's actual data	A rental object with known values	Returns a dictionary with all six fields matching the object's current attributes exactly